

• Intrări:

- Modele geometrice
- Modele de iluminare
- Descrierea scenei

• Motor grafic:

- Transformări
- Proiecții
- Rasterizare
- Iluminare
- Umbrire

• Texturi:

Imagini

• Prezentare:

Dispozitiv de afișare

Aplicație → Procesare geometrică → Rasterizare → Procesarea pixelilor

- etapele se exec. în paralel, fiecare fiind dependentă de rezult. celei de dinainte.

Pipeline-ul grafic → generează (trasează) o imagine 2D

①. Aplicația

- se exec. de obicei pe CPU
- trimite către GPU (câtre etapa de procesare geometrică) primitivele grafice ce vor fi trasate:

- Puncte, Linii, Triunghiuri

- se exec. în paralel, pe mai multe nuclee ale CPU:
 - detecția coliziunilor
 - alg. globale de accelerare
 - simulări fizice
 - procesarea intrărilor utilizator

②. Procesarea geometrică - calc. ce trebuie desenat, cum, pe unde
→ majoritatea operațiilor / vârf și / triunghi

1) Vertex Shading - coord. model $\xrightarrow{\text{transform}}$ world coord (world space)

- calc. poz. vârfurilor
 - ~ Spațiul obiect
 - ~ Spațiul scenă
 - ~ Spațiul de vizualizare

- calc. valorilor adiționale per-vârf
 - ~ Normale
 - ~ Coordonate textură
 - ~ Culoare

rezultatele trimise spre departe plane de decupare

- camera: poz și direcție (world space) $\xrightarrow{\text{view transform}}$ view space (sp. de vizualizare)

2) Proiecția

→ proiectarea scenei din 3D în 2D

- proiecții $\left\{ \begin{array}{l} \text{paralele} \text{ (linii || rămân || după transform.)} \\ \text{perspectivă} \end{array} \right.$ (↑ pt a ușura sp. de proiecție și decupare)

- coordonate de decupare
 - ~ Spațiul de decupare
 - ~ Împărțirea de perspectivă
 - ~ Coord. de vizualiz. normalizate

3) Decuparea (Clipping) - vârf pe care vârf le vede camera

→ parțială sau totală a primitelor care cad în afara ferestrei

- se folosește o matrice de proiecție

4) Maparea pe ecran - coord sunt încă 3D la intrarea în etapă

→ calc. pozițiilor vârfurilor pe ecran

- ~ Spațiul ecran (window coord)

Procesare geometrică opțională:

• Teselare

- îmbunătățirea detaliilor la frontiere
- redimensionarea setului de vârfuri
- generarea de noi triunghiuri

• Geometry shading

- generarea de geom. adițională
- gen. particulelor

③. Rasterizarea (Rasterization) - ia ca input 3 ef. (Δ) și găsește toți pixelii din int. lui

1) Configurarea triunghiurilor

- se calc. parametrii din ec. laturilor și ec. derivate ale laturilor

2) Parcursirea triunghiurilor

- se verif. pixelii acoperiți de triunghi
- se generează un fragment pt. fiecare pixel acoperit din ef. 2D în screen space → pixeli pe ecran

④. Procesarea pixelilor

1) Pixel shading

- calc. culorilor pixelilor
- ~ Modele de iluminare
- ~ Texturare
- ~ Umbrire

- informația pt. fiecare pixel este stocată în bufferul de culoare

2) Combinarea fragmentelor

- combinarea culorilor fragm. în bufferul de culoare
- rezolvarea probl. de vizibilitate → z-buffer
- blending (ef. de transparență)
- Stencil buffer, ef. speciale

- frame buffer-ul e alc. din toate bufferele dintr-un sistem

UNITATEA DE PROCESARE GRAFICĂ (GPU) 30-56

→ procesare hardware a vârfurilor

Initial → pipeline GPU configurabil
(funcționalit. fixă)

Uterior → programabil
(funcț. programabilă)

Shadere programabile

= procesare mici care realiz. sarcini relativ izolate

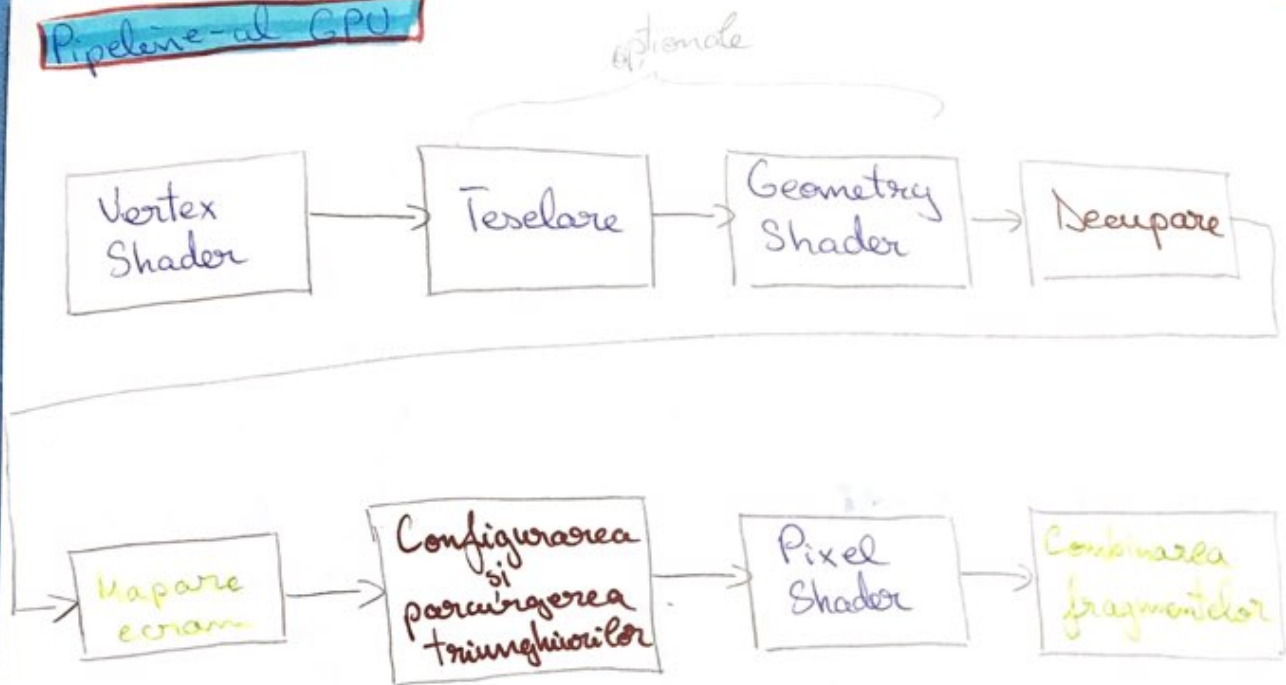
• Unitatea de procesare de bază: shaderul

• Instantele care rulează în shader: threaduri

↳ mapate pe benzi SIMD

• SIMT (Single Instn. Multiple Threads)

Pipeline-ul GPU



= { programabil
configurabil
funcționalitate fixă

Pipeline-ul GPU programabil

- shaderele de vârfuri, teselare, geometrie și pixeli au model de programare comun
- același set de instrucțiuni (ISA)

→ avantaje:

echilibrarea încărcării vertex/pixel shaders

Variabile de intrare shader:

- **uniform** - constante pe tot parcursul unei instruct. de desen
- **varying** - valori diferite pt. fiecare instanță de shader
- **texture** - var. uniform specială

vin de la vârfurile triunghiului
sau de la rasterizare

Shaderul de vârfuri

→ operații per-vârf

- culoare
- poziție
- normală
- coord. textură

} - trimese mai departe
pt. interpolare

→ transformă pozițiile vârfurilor

din spațiul **obiect** → sp. de **decupare**

→ **fiecare vf. este procesat independent**

- pot rula în paralel oricâte procesare shader

→ vertex blending, skinning sau morfism, deformarea meshurilor,
hăști de adâncime, modelarea texturilor sau a apei

Texturarea

- ajută la **trazarea supraf. curbate**

- crearea de noi vf. și pt. o mai bună aproximare a curbelor

avantaje:

• **descrierea supraf. curbate**

- mai compactă decât aproximarea prin triunghiuri

• **rezoluția supraf. curbate**

- ajustată în funcție de **niv. de detaliu necesar**

Shaderul de geometrie

→ poate transforma primitive în alte primitive

- date de intrare:

un obiect (vârfuri / segmente / triunghiuri)

- date de ieșire:

zero sau mai multe vârfuri

→ mapare cubică, algoritmi avansați de umbruire, crearea
particulelor de dimensiuni var. din pte, pt. trazarea firelor de păr,
identif. conturului obiectelor pt. alg. de umbruire
(ex: umbre volumetrice)

Shaderul de pixeli (fragment shader)

- procesează fragmentele care rezultă în urma rasterizării
- date de intrare:
 - datele de ieșire ale vertex shader-ului, interpolate
- date de ieșire:
 - culoarea fragmentului procesat
- fog computation
- poate respinge fragmente (discard)

Shaderul de calcul

- nu este parte din pipeline-ul grafic, dar este legat de pipeline
- elimină transferul CPU-GPU
- memorie partajată între thread-uri
- blur, ocean waves

Combinarea fragmentelor

- etapa în care adâncimile și culorile fragm. individuale generate în pixel shader sunt combinate cu frame buffer-ul
- aici au loc operațiile stencil-buffer și z-buffer și color

blending

- indiferent de ordinea în care rezultatele din pixel shader sunt generate, ele sunt sortate și trimise către etapa de combinare în ordinea de intrare, obiect cu obiect și Δ cu Δ .

CURS 2

Math

Vectori → lungime, direcție

- geometrie: răgeti în spațiu
- algebra: arrays of numbers

$$v = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$u = k \cdot v \Rightarrow u, v$ - paraleli → dependenți linear

- vect. non-paraleli $\Rightarrow$ independenți linear $\Rightarrow$ bază 2D

$\text{span}(u, v) = a_1 u + a_2 v$, $\forall a_1, a_2$ - numere

- 2 vectori formează o bază ortonormală dacă:

$$2D \begin{cases} \cdot \text{sunt ortogonali: } x \cdot y = 0 \\ \cdot \text{sunt vect. unitate: } \|x\| = \|y\| = 1 \end{cases}$$

$$3D \begin{cases} x \cdot y = y \cdot z = z \cdot x = 0 \\ \|x\| = \|y\| = \|z\| = 1 \end{cases}$$

lungimea

$$\text{length}(v) = \|v\| = \sqrt{x^2 + y^2}$$

- zero vector: $\|v\| = 0$

- unit vector: $\|v\| = 1$

normalizare = convertim un vect. la un vect. unitate

$$v_{\text{unit}} = \frac{v}{\|v\|}$$

Dot product (= produs scalar)

$$u \cdot v = u_x v_x + u_y v_y$$

$$u \cdot v = \|u\| \cdot \|v\| \cdot \cos \theta$$

• dist. dintre 2 pte:

$$\theta = Q - P$$

$$\text{dist}(P, Q) = \text{length}(v) = \|v\| = \sqrt{v \cdot v}$$

- unghiul dintre 2 vect.:

$$\theta = \cos^{-1} \frac{u \cdot v}{\|u\| \cdot \|v\|}$$

$$u \cdot v < 0 \Rightarrow \theta > 90^\circ$$

$$u \cdot v = 0 \Rightarrow \theta = 90^\circ$$

$$u \cdot v > 0 \Rightarrow \theta < 90^\circ$$

- poz. relativă a unui pct. față de un plan:

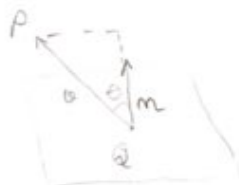
$$\theta > 90^\circ \Rightarrow P \text{ e în fața planului}$$

$$\theta = 90^\circ \Rightarrow P \text{ e pe plan}$$

$$\theta < 90^\circ \Rightarrow P \text{ e în spatele planului}$$

- dist. dintre un punct P și un plan

$$\|w\| = \|v\| \cdot \cos \theta = \|v\| \cdot \frac{n \cdot v}{\|n\| \cdot \|v\|} = \frac{n \cdot v}{\|n\|}$$



Cross product (= product vectorial)

$$\|v \times w\| = \|v\| \|w\| \sin \theta$$

$$\begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} \times \begin{bmatrix} w_x \\ w_y \\ w_z \end{bmatrix} = \begin{bmatrix} v_y w_z - v_z w_y \\ v_z w_x - v_x w_z \\ v_x w_y - v_y w_x \end{bmatrix}$$

$$v \times w = -w \times v$$

- calc. vectorului normală

$$n = e_1 \times e_2, \text{ unde } e_1 \text{ și } e_2 \text{ sunt 2 muchii consec. ale unui poligon}$$

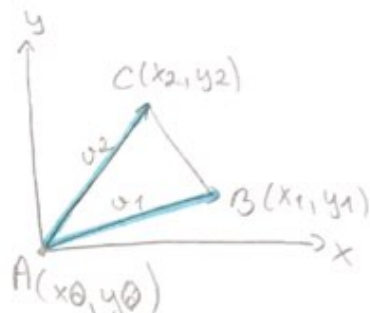
- n poartă în afara ab
 $\Rightarrow$ counterclockwise

- n poartă în int. ab.
 $\Rightarrow$ clockwise

• aria unui Δ

$$v = v_1 \times v_2 = \begin{vmatrix} e_1 & e_2 & e_3 \\ x_1 & x_2 & 0 \\ y_1 & y_2 & 0 \end{vmatrix}$$

$$v = (x_1 y_2 - x_2 y_1) e_3$$



$$\begin{aligned} A_{\Delta ABC} &= \frac{1}{2} (x_1 - x_2)(y_1 + y_2) + \\ &\quad \frac{1}{2} (x_2 - x_0)(y_2 + y_0) + \\ &\quad \frac{1}{2} (x_0 - x_1)(y_0 + y_1) \\ &= \frac{1}{2} (x_0 y_1 - x_1 y_0 + x_1 y_2 - x_2 y_1 + x_2 y_0 - x_0 y_2) \\ &= \frac{1}{2} (x_1 y_2 - x_2 y_1) \\ &= \frac{1}{2} \|v_1 \times v_2\| \end{aligned}$$

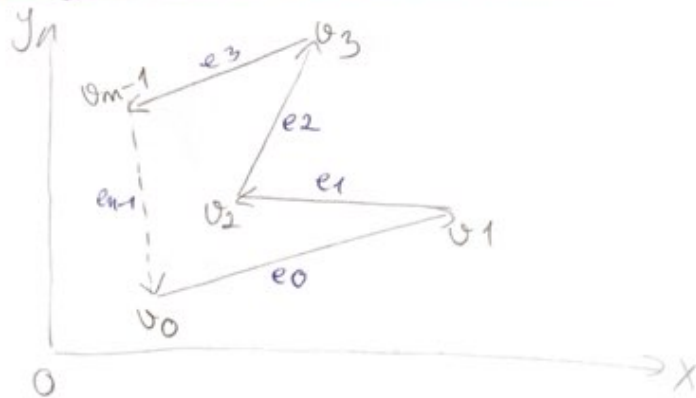
• aria unui poligon

$$S = \frac{1}{2} \sum_{i=0}^{n-1} \|v_i \times v_{i+1}\|$$

Poligoane convexe sau concave

- prod. vectorial al muchilor consecutive
 - schimbarea direcției $\Rightarrow$ concave
 - același sens $\Rightarrow$ convexe
- cel puțin o intersecție între o muchie și poligon $\Rightarrow$ concav

Testare polig. convexe / concave 2D



- transformăm polig. pe planul xy
- calc. vect. muchilor în ordinea inversă sensului acelor de ceasornic

ex: $e_1 = v_2 - v_1$

- pt. fiecare vect. muchie, setăm componenta $z=0$
 - calc. prod. vectorial pt. fiecare pereche de muchii consec.
- același semn $\Rightarrow$ convex
 - semne diferite $\Rightarrow$ concav

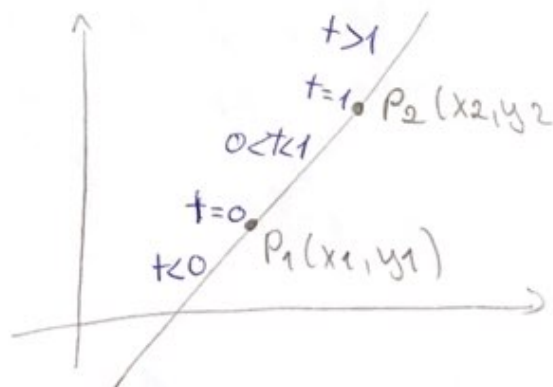
LINE

$y = mx + b$, $m \rightarrow$ panta, pct. $(0, b) \rightarrow$ intersecția cu axa y

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

Parametric line

$$P = P_1 + t(P_2 - P_1)$$



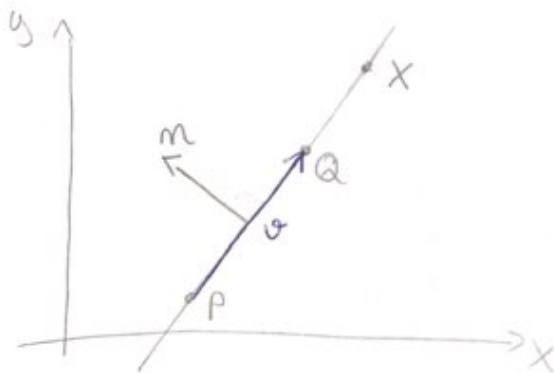
Implicit line

$$(X-P) \cdot n = 0$$

$$v = Q - P$$

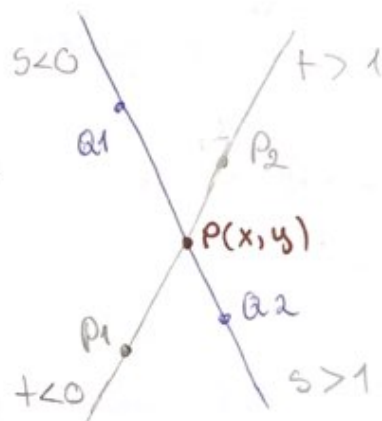
$$= \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

$$n = \begin{bmatrix} -v_y \\ v_x \end{bmatrix}$$



Intersecție 2 linii

$$P = (1-t)P_1 + tP_2 = (1-s)Q_1 + sQ_2$$



$t < 0, s < 0 \Rightarrow P$ înainte de P_1 și Q_1

$0 < t < 1, 0 < s < 1 \Rightarrow P_1P_2$ intersecțiază Q_1Q_2

$t > 1, s > 1 \Rightarrow P$ după P_2 și Q_2

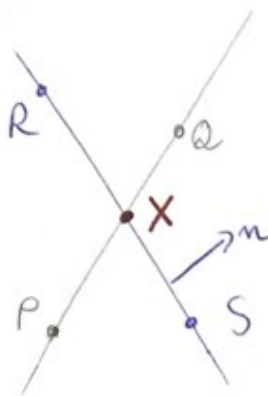
$$X = (1-t)P + tQ$$

$$(X-S) \cdot n = 0$$

$$u = Q - P$$

$$v = R - S$$

$$t = \frac{-v \cdot n}{u \cdot n}$$



Relatia dintre puncte

$$D = \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}$$

$P(x_1, y_1)$

$Q(x_2, y_2)$

$R(x_3, y_3)$

$D > 0 \Rightarrow$ counterclockwise

$D = 0 \Rightarrow$ pte coliniare

$D < 0 \Rightarrow$ clockwise

Relatia dintre un pct. si un triunghi $\triangle ABC$, pct P

• in int. Δ :

$$\text{sign}(A, B, P) = \text{sign}(B, C, P) = \text{sign}(C, A, P)$$

$(x, y) \cdot$

• altfel $\Rightarrow$ in afara Δ

Relatia dintre un pct. si un poligon

- P in int. polig. daca:

- suma $\angle$ dintre muchii = 360°

- P e in int. cel puțin al unui Δ

- nr. de Δ dintre o linie din P si $\Delta \rightarrow$ impar

Relatia dintre un pct. si un plan

- plan def. de ptele A, B, C

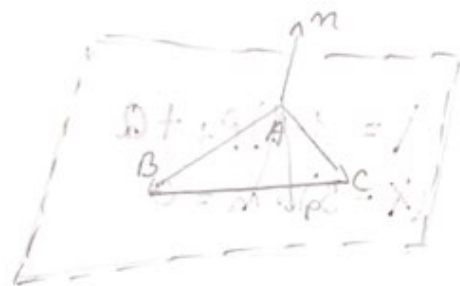
$$n = (B-A) \times (C-A)$$

$$f(P) = n \cdot (P-A) = 0$$

$f(P) = 0 \Rightarrow P$ e pe plan

$f(P) > 0 \Rightarrow P$ e pe aceeași parte cu n

$f(P) < 0 \Rightarrow P$ e pe partea opusă de n



Obiecte 3D convexe / concave

Intersecția linie-plan

Ec. liniei : $L(t) = P + (Q - P)t$

Ec. planului : $(Y - A) \cdot n = 0$

$$(P + (Q - P)t - A) \cdot n = 0$$

$$((P - A) + (Q - P)t) \cdot n = 0$$

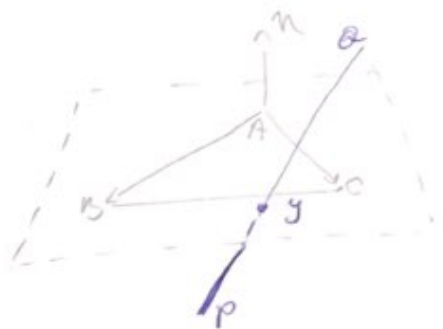
$$(P - A) \cdot n + t(Q - P) \cdot n = 0$$

$$t = \frac{(A - P) \cdot n}{(Q - P) \cdot n}$$

- punctul de intersecție $Y = L(t)$

Intersecția dintre un Δ și un plan

- verif. dacă vreun vf. al Δ e pe partea opusă de celălalt
- găsim puncte de intersecție unde planul taie Δ
- generăm 3 noi Δ



CURS 3

Transformări

Transformări geom. de bază

①. Liniare $\rightarrow$ scalarea și rotația 3×3

②. Afine $\rightarrow$ translația 4×4

$\rightarrow$ păstrează paralelismul liniilor

$\rightarrow$ nu păstrează neapărat lungimile și unghiurile

TRANSLATIA

- schimbarea locației

- deplasează o entit. după un vector $t = (t_x, t_y, t_z)$

$$T(t) = T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$p = (p_x, p_y, p_z, 1)$$

$$p \rightarrow p' = T(t)p$$

$$p' = (p_x + t_x, p_y + t_y, p_z + t_z, 1)$$

inversa:

$$T^{-1}(t) = T(-t)$$

ROTAȚIA

- rotește un vector cu un unghi dat în jurul unei axe care trece prin origine

$$\text{Punctul } v = (v_x, v_y) = r \cdot \cos \theta, r \cdot \sin \theta$$

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_y(\alpha) = \begin{pmatrix} \cos \alpha & 0 & \sin \alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \alpha & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

inversa:

$$R_i^{-1}(\alpha) = R_i(-\alpha)$$

Rotatie in jurul unui punct arbitrar:

$T(-c)$

R_z

$T(c)$

SCALARE

$$S(s) = S(s_x, s_y, s_z)$$

$$S(s) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

inversa:

$$S^{-1}(s) = S\left(\frac{1}{s_x}, \frac{1}{s_y}, \frac{1}{s_z}\right)$$

- 1 sau 3 val. negative :
→ matrice de reflexie
- 2 val. negative $\Leftrightarrow$ rotație cu π radiani

FORFECAREA (shearing)

= deformarea obiectului de-a lungul unei axe în funcție de o altă axă și parametrul s

→ $H_{xy}(s), H_{xz}(s), H_{yx}(s), H_{zx}(s), H_{yz}(s), H_{zy}(s)$

→ $H_{ij}(s)$ $\begin{cases} i - \text{coord. care se modif.} & (\text{linia}) \\ j - \text{coord. în funcție de care se modif.} & (\text{col}) \end{cases}$

$$H_{xz}(s) = \begin{pmatrix} 1 & 0 & s & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

inversa:

$$H_{ij}^{-1}(s) = H_{ij}(-s)$$

Concatenarea transformărilor

ad: $S \rightarrow R \rightarrow T$

$$TRS_p = (T(R(Sp)))$$

Transformări rigide

- nu modif. forma obiectelor
- translații și rotații

$$X = T(H)R = \begin{pmatrix} r_{100} & r_{101} & r_{102} & t_x \\ r_{110} & r_{111} & r_{112} & t_y \\ r_{120} & r_{121} & r_{122} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Orientarea camerei

- baza $\{n, u, v\}$

$$M = \begin{pmatrix} n_x & n_y & n_z & -t \cdot n \\ u_x & u_y & u_z & -t \cdot u \\ v_x & v_y & v_z & -t \cdot v \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Transformări geom. speciale

Transformarea Euler

- metoda intuitivă de a construi o matrice de orientare
unghiurile lui Euler:

- head (yaw) $\rightarrow h$ y
- pitch $\rightarrow p$ x
- roll $\rightarrow r$ $-z$

$$E(h, p, r) = R_z(r) R_x(p) R_y(h)$$

Gimbal lock

- se pierde un grad de libertate $\Rightarrow$ celelalte 2 rotații se fac după aceeași axă
- h și r se rotesc în jurul aceleiași axe $\Rightarrow h = 0$ (una dintre ele)

QUATERNIONI

- extensie a numerelor complexe
- utiliz. pt. a reprez. rotații și orientări
- superioare transformărilor Euler sau matriceale
- pot fi utiliz. pt. interpolare constantă și stabilă a orientărilor

$$\hat{g} = (g_0, g_w) = i g_x + j g_y + k g_z + g_w$$

g_w - parte reală

g_x - parte imaginară

$\rightarrow$ operații vec.

$$i^2 = j^2 = k^2 = -1$$

$\hat{q}(q_v, q_w)$ - cuaternion unitate $\Leftrightarrow n(\hat{q}) = 1$

Punct $p = (p_x, p_y, p_z, p_w)^T \Rightarrow$ cuaternion $\hat{p}$

$\hat{q} = (\sin \alpha u_g, \cos \alpha)$ - cuaternion unitate

$$\hat{q}^{-1} = \hat{q}^*$$

Interpolare liniară sferică

$\hat{q}$ și $\hat{n}$ - cuaternioni unitate

param. $t \in [0, 1]$

Cuaternionul interpolat $\hat{s}$

(cel mai scurt arc pe o sferă 4D de la $\hat{q}$ la $\hat{n}$)
($t=0$) ($t=1$)

$$\hat{s}(\hat{q}, \hat{n}, t) = \text{slerp}(\hat{q}, \hat{n}, t)$$

$$= \frac{\sin(\alpha(1-t))}{\sin \alpha} \hat{q} + \frac{\sin(\alpha t)}{\sin \alpha} \hat{n}$$



VERTEX BLENDING

- tehnică utilizată pt. animarea încheieturilor flexibile
- suprapunerea transformărilor rigide
- pp. interconectarea porțiunilor fixe (atașate unor oase) cu porțiuni deformabile (model deformabil - "skin")
- = transformarea triunghiurilor (vârfurilor) porțiunii flexibile cu mai multe matrici model - matr. model ale oaselor adiacente

MORFISM

= transformarea unui model geometric în altul prin interpolare în timp ($t_0 \rightarrow t_1$)

- pb. majore:

- corespondența vârfurilor
- problema interpolării

Dacă $\exists$ corespondență 1-1 între vârfuri $\Rightarrow$ interpolare liniară

$$s = (t - t_0) / (t_1 - t_0)$$

$$m = (1-s)p_0 + sp_1$$

p_0, p_1 - vârful p la mom. de timp t_0, t_1

ex.: model transf. prin morfism: face (morph targets / blend shapes)
neutral $\longrightarrow$ smiling

$\rightarrow$ skinning and morphing

$\rightarrow$ vectori: diferență
 $\hookrightarrow$ to add movements on the neutral face

PROIECTII

- perspectivă: - fotorealism (cum văd oamenii)
 - raze de proiecție converg către centru (locul privitorului)
 - ob. par mai mici odată cu adâncimea
 - liniile care nu sunt paralele cu planul de proiecție converg către un rg. punct
- paralela:
 - raze de proiecție $\parallel$
 - liniile $\parallel$ rămân $\parallel$
 - poz. privitorului, ptul de convergență $\rightarrow$ la ∞

ortogonală :

$$P_0 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

↑
nu este inversabilă

$$z = 0$$

perspectivă :

- camera centrată în origine

$$P_p = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1/d & 0 \end{pmatrix}$$

85-93 (note)

CURS 4

Reprezentarea și modelarea obiectelor 3D

①. Funcții implicite

②. Reprezentare poligonată

• avantaje:

- omni prezentă în grafica pe calculator
- simplă și directă
- folosită ca formă intermediară pt. multe alte struct. de date

• probleme:

- aproximarea supraf. curbe
- precizia depinde ft. mult de nr. de poligoane
- maparea texturii
- suprafețe netede
- alg. de umbrire

→ Boundary representation:

• Brep

ex.: Polygon (face) | Vertices
p1 v1v2v3v4

- dezavantaje:

- muchiile sunt împartite între polig. adiacente
- o muchie e procesată de 2 ori

• Brep-edge based

Polygon | Edges

Edges | Vertices

Triangulation

- > first choice of polygons
- > simple and fast algorithms

Reprezentare Δ :

- lists - 3 of Δ
- Fans - 1.5 of Δ
- Strips - cea mai eficientă reprez
1.5 of Triunghi

③ Bicubic parametric patches

$$P(u,v) = [x(u,v), y(u,v), z(u,v)]$$

$$0 \leq u \leq 1, 0 \leq v \leq 1$$

- curbe:
Hermite, Bezier, Spline, Uniform B-Spline
- suprafețe:
Hermite, Bezier, B-Spline

Avantaje:

- exactă și economică, pt. că e analitică
- deformările suprafețelor parametrice apar netede

④ Constructive Solid Geometry (CSG)

-> scalare, rotație, translație

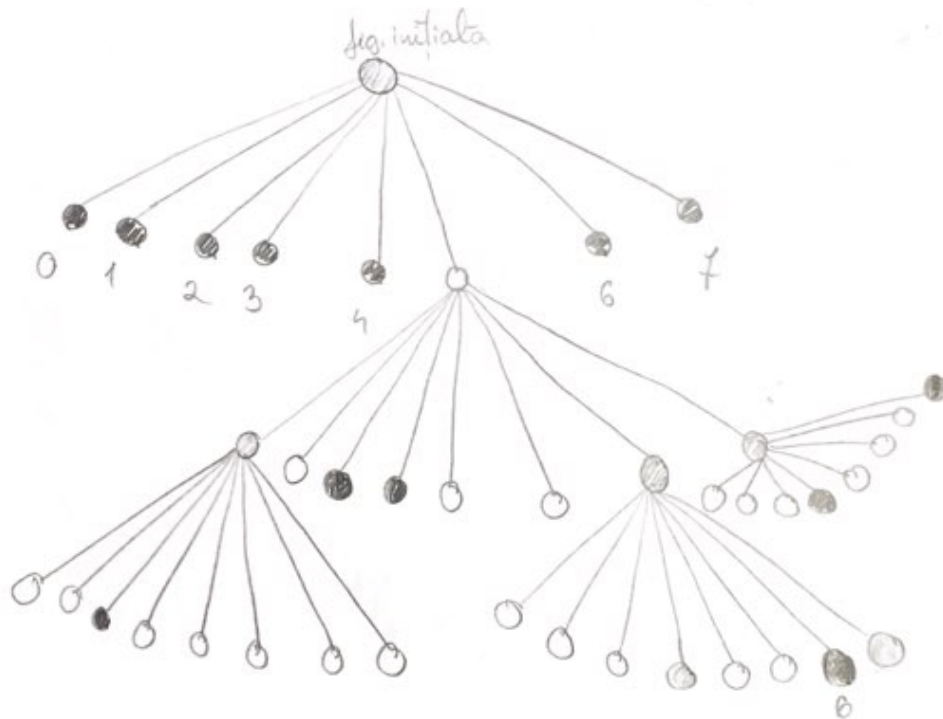
- modelare ierarhică bazată pe struct. de arbore;
 - nodurile interne: union, intersection sau difference
 - the edges of the tree au transf. asociate cu ele:
scalare, rotație, translație
 - frunzele conțin doar geometrie
- permite forme complexe cu doar câteva primitive
 - cilindri, cuburi, supraf. cuadratice etc.
- motivated by CAD and manufacture
 - op. booleani corespund operațiilor mecanice

Tehnici de subdiviziune a spațiului:

• **Octree representation** (slide 23) → cuburi care nu se intersect.

- am o figură (cub)
- împart în 8 și apoi tot împart unde e nevoie (0...7)
- arbore:

- negru - cubul există în fig.
- gri - partial (descompus în alte 8 cuburi)
- alb - doar numerotat, nu există în figură



0 - jos - fata - stg.
1 - jos - fata - dr.
2 - jos - spate - dr.
3 - jos - spate - stg.

4 - sus - fata - stg.
5 - sus - fata - dr.
6 - sus - spate - dr.
7 - sus - spate - stg.

→ pt. numerotarea back-front: (tabel)

$x + 1 -$ (dr / stg)

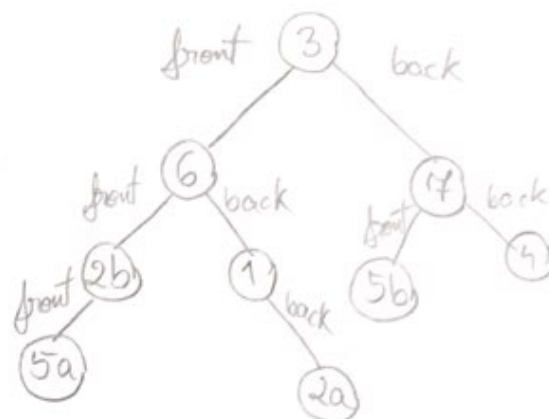
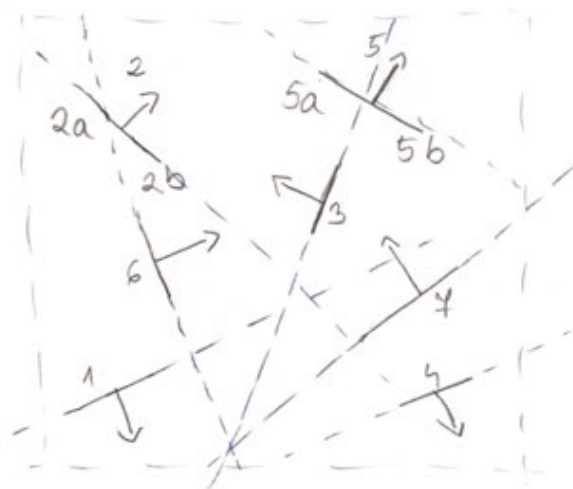
$y + 1 -$ (sus / jos)

$z + 1 -$ (fata / spate)

} vizibilit. fețelor

• BSP Trees

- ajung la desenul cu normalele: (nu ştiu cum încei)



- încep de la 3 (cam mijloc)

↳ dg: front - parte cu normala
↳ dr: back

Modelare procedurală

Sisteme bazate pe particule

ex.: iarbă, cascade, artificii etc

Particle life Cycle

- ①. Generare - generate aleator într-o locație predeterminată a ob. fuzzy
- ②. Dinamica particulelor
- ③. Extindere - fiecare particulă are 2 atribute: age, lifetime
- ④. Rendering - trib. fațete mixte asumptii pt. a simplifica procesul

Mass-spring

↳ cloth
↳ foratori (sewing)